HIGH-LEVEL DESIGN (HLD) DOCUMENT

1. SYSTEM OVERVIEW

1.1 Problem Definition:

Customer support systems often rely on manual processes or rule-based chatbots that provide generic responses and lack the ability to access and interpret domain-specific documents. This leads to inefficiencies, increased response times, and inconsistent answer quality. Additionally, traditional systems struggle to handle large volumes of queries and fail to deliver context-aware responses based on organizational knowledge bases such as policy documents or FAQs.

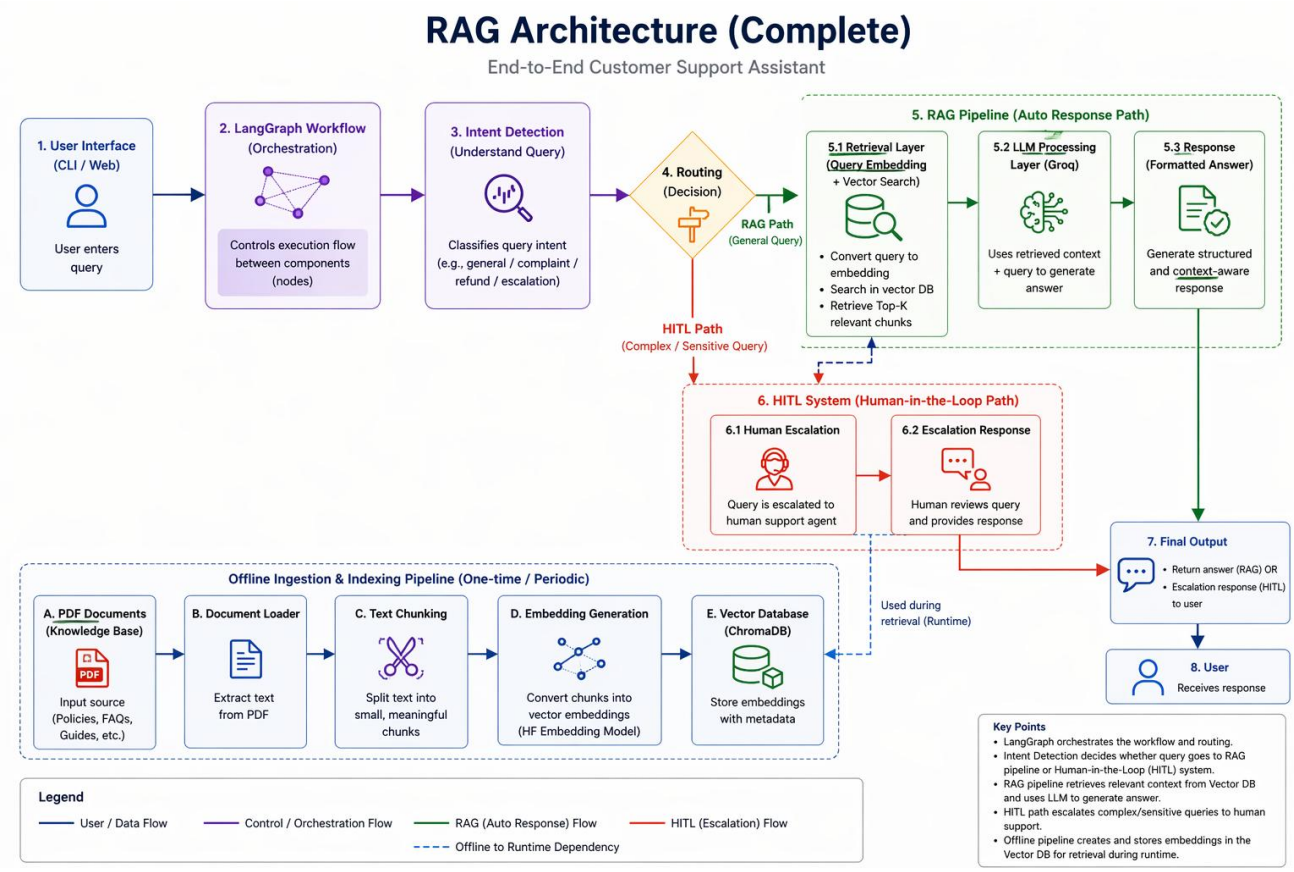
To address these limitations, there is a need for an intelligent system that can automatically retrieve relevant information from structured documents and generate accurate, context-driven responses while maintaining the ability to escalate complex queries to human agents when necessary.

1.2 Scope of the Project:

The system is designed to process a PDF-based knowledge base, extract and segment its content, and convert it into embeddings stored within a vector database for efficient retrieval. It supports natural language query input and retrieves relevant document segments to generate context-aware responses using a large language model.

The system uses a graph-based workflow to manage how tasks are performed, such as understanding the user's intent, generating responses, and deciding the next step. It also includes a Human-in-the-Loop (HITL) feature, which means that complex or sensitive queries are passed to a human for better handling. The system is limited to answering questions based only on the given PDF knowledge base and does not use external data or support long conversations beyond a single query.

2. ARCHITECTURE DIAGRAM



This RAG architecture is split into an offline ingestion pipeline and an online query pipeline. In the offline stage, documents like PDFs are loaded, chunked, converted into embeddings, and stored in a vector database (ChromaDB). This builds a searchable knowledge base. In the online stage, a user query enters through the interface and is processed by LangGraph orchestration and intent detection. Based on the query type, it is either routed to a RAG pipeline or escalated to a human (HITL). In the RAG path, the query is embedded and used to retrieve relevant chunks from the vector database. The retrieved context is sent to an LLM to generate a response. Finally, the system returns a formatted answer to the user.

3. COMPONENT DESCRIPTION

3.1 Document Loader

The Document Loader is responsible for reading the PDF knowledge base and extracting its textual content. It converts the document into a structured format that the system can process further. This step is essential because raw PDFs are not directly usable by machine learning models. By loading and extracting the text, the system prepares the data for downstream processing and ensures that all relevant information from the document is available for retrieval.

3.2 Chunking Strategy

The Chunking Strategy divides the extracted text into smaller, meaningful segments called chunks. This is necessary because large documents cannot be processed efficiently as a whole, and important information may get diluted. By splitting the text into smaller parts, the system improves retrieval accuracy and ensures that only the most relevant sections are selected during querying. Overlapping between chunks is often used to maintain context continuity between segments.

3.3 Embedding Model

The Embedding Model converts each text chunk into a numerical vector representation. These embeddings capture the semantic meaning of the text, allowing the system to understand relationships between words and sentences beyond simple keyword matching. This is important because user queries may not use the exact same words as the document, but the embedding model can still identify similar meanings. This step enables efficient and intelligent semantic search.

3.4 Vector Store

The Vector Store is responsible for storing all the generated embeddings in an organized and searchable format. In this project, ChromaDB is used as the vector database. It allows fast similarity-based retrieval by comparing vector distances. This component is crucial because it acts as the system’s memory, enabling quick access to relevant information when a query is received.

3.5 Retriever

The Retriever is used to fetch the most relevant chunks from the vector store based on the user's query. It converts the query into an embedding and compares it with stored vectors to find the closest matches. This ensures that only the most contextually relevant information is selected. The retriever improves the quality of responses by providing accurate and focused context to the language model.

3.6 LLM (Large Language Model)

The Large Language Model (LLM) generates the final response using the user's query and the retrieved context. It combines its language understanding capabilities with the provided information to produce clear and meaningful answers. The LLM is important because it transforms raw retrieved data into human-readable responses, making the system interactive and user-friendly.

3.7 Graph Workflow Engine

The Graph Workflow Engine controls the overall execution flow of the system. In this project, LangGraph is used to define and manage the sequence of operations. It organizes the process into nodes and transitions, allowing structured and flexible execution. This component is important because it enables modular design, better control, and conditional decision-making within the system.

3.8 Routing Layer

The Routing Layer determines how a user query should be handled. It analyzes the intent of the query and decides whether it can be answered using the RAG pipeline or needs escalation. This decision-making step is important because not all queries are suitable for automated responses. By routing queries appropriately, the system improves accuracy and reliability.

3.9 HITL Module (Human-in-the-Loop)

The Human-in-the-Loop (HITL) Module handles cases where the system cannot confidently generate a response. In such situations, the query is escalated to a human expert for resolution. This ensures that critical, ambiguous, or sensitive queries are handled correctly.

The HITL module is essential for maintaining trust, reducing errors, and making the system suitable for real-world customer support applications.

4. DATA FLOW

4.1 From PDF to Answer

The data flow in the RAG-based Customer Support Assistant begins with the PDF knowledge base and ends with a generated response to the user. Initially, the PDF is processed by the Document Loader, which extracts the textual content from the file. This raw text is then passed through a chunking process, where it is divided into smaller, meaningful segments to improve processing efficiency and retrieval accuracy.

These chunks are converted into vector representations using an embedding model, which captures the semantic meaning of the text. The generated embeddings are then stored in a vector database, specifically ChromaDB, where they are indexed for efficient similarity search. At this stage, the system has transformed unstructured document data into a structured and searchable format.

When a user submits a query, the system follows a retrieval process. The query is first converted into an embedding and compared with the stored vectors in the database. The most relevant chunks are retrieved based on similarity scores. These retrieved pieces of information act as contextual input for the Large Language Model (LLM), which generates a final answer that is both accurate and contextually grounded. This entire process ensures that responses are based on actual document knowledge rather than relying solely on the model's internal memory.

4.2 Query Lifecycle

The query lifecycle describes how a user query is processed step by step within the system. It starts when the user enters a query, which is received by the system's input interface. The query is first analyzed by the routing layer to determine the user's intent. Based on this analysis, the system decides whether the query can be handled using the RAG pipeline or if it requires escalation.

If the query is suitable for automated answering, it is passed through the retrieval process. The system converts the query into an embedding and uses it to fetch relevant information from the vector store. This retrieved context is then combined with the original query and sent to the LLM for response generation. The generated answer is then returned to the user as the final output.

However, if the routing layer determines that the query is complex, ambiguous, or requires human intervention, the system activates the Human-in-the-Loop (HITL) module. In this case, the query is escalated to a human agent instead of being answered automatically. This ensures that the system maintains reliability and avoids incorrect or misleading responses.

Overall, the query lifecycle ensures that every user request is handled efficiently, either through intelligent retrieval and generation or through appropriate escalation when necessary.

5. TECHNOLOGY CHOICES

Why ChromaDB

ChromaDB is used as the vector database in this project because it is specifically designed for storing and retrieving embeddings efficiently. It provides fast similarity search, which is essential for retrieving relevant document chunks based on user queries. One of the key reasons for choosing ChromaDB is its simplicity and ease of integration with frameworks like LangChain. It is lightweight, does not require complex setup, and works well for small to medium-scale applications. Additionally, it supports persistent storage, allowing the system to reuse previously created embeddings without recomputing them every time. This makes it a practical and efficient choice for implementing the retrieval component of a RAG system.

Why LangGraph

LangGraph is used to design and manage the workflow of the system in a structured and flexible way. Unlike traditional linear pipelines, LangGraph allows the system to be modeled as a graph with nodes and conditional transitions. This makes it ideal for handling complex flows such as intent-based routing and Human-in-the-Loop (HITL) escalation. The ability to define conditional paths enables the system to dynamically decide whether to proceed with automated response generation or escalate the query. This level of control improves

modularity, scalability, and maintainability of the application, making LangGraph a suitable choice for orchestrating the entire RAG pipeline.

LLM Choice (Large Language Model)

A Large Language Model (LLM), typically accessed through OpenAI APIs in this project, is used for generating natural language responses. The reason for choosing an LLM is its strong ability to understand context and generate human-like answers. Instead of relying on predefined responses, the LLM uses the retrieved document context to produce accurate and coherent answers. This significantly improves the quality of customer support interactions. Additionally, using a pre-trained LLM eliminates the need for training a model from scratch, saving time and computational resources while still achieving high performance.

Additional Tools

Several supporting tools are used to enhance the system's functionality. **LangChain** is used to integrate different components such as document loading, embeddings, and retrieval in a unified framework. It simplifies the development process by providing pre-built modules for RAG pipelines. The **pypdf** library is used for extracting text from PDF documents, enabling the system to process the knowledge base. The **tiktoken** library helps in managing token limits, ensuring that the input to the LLM stays within acceptable boundaries. Together, these tools streamline the implementation and ensure that the system operates efficiently and reliably.

6. SCALABILITY CONSIDERATIONS

6.1 Handling Large Documents

When dealing with large documents, the system must ensure efficient processing without losing important context. This is addressed through an effective chunking strategy, where the document is divided into smaller, manageable segments before generating embeddings. To maintain context continuity, overlapping chunks can be used so that important information is not split across boundaries. Additionally, embeddings are stored in a vector database like ChromaDB, which allows efficient indexing and retrieval even when the number of chunks grows significantly. For very large datasets, techniques such as hierarchical chunking or document summarization can be applied to reduce storage overhead and improve retrieval performance.

Increasing Query Load

As the number of user queries increases, the system must be able to handle multiple requests efficiently without degradation in performance. This can be achieved by scaling the application horizontally, such as deploying the system as an API using frameworks like FastAPI and running it behind a load balancer. Caching frequently asked queries and their responses can significantly reduce repeated computation and improve response time. Additionally, optimizing the retriever to fetch only the top relevant chunks (top-k retrieval) helps reduce unnecessary processing. Proper resource management and asynchronous processing also ensure that multiple queries can be handled concurrently without bottlenecks.

Latency Concerns

Latency is a critical factor in user experience, especially in customer support systems where quick responses are expected. The main contributors to latency are embedding generation, vector search, and LLM response time. To reduce latency, embeddings should be precomputed during the indexing phase rather than at query time. Efficient similarity search in ChromaDB helps minimize retrieval delay. Additionally, limiting the amount of context sent to the LLM and optimizing prompt size can reduce response generation time. In production systems, techniques such as response streaming, caching, and using faster or smaller LLM variants can further improve responsiveness.